

Customer Segmentation using Boltzmann Machine (RBM)

Subject: ANNDL

Name: Shruti Kusumbe A-57

Class: TY-A AI(AI)DS

Aim

To perform customer segmentation using a Restricted Boltzmann Machine based on online retail shopping data.

```
# Install required package
!pip install openpyxl

# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.neural_network import BernoulliRBM
from sklearn.cluster import KMeans

Requirement already satisfied: openpyxl in /usr/local/lib/python3.12/dist-packages (3.1.5)
Requirement already satisfied: et-xmlfile in /usr/local/lib/python3.12/dist-packages (from openpyxl) (2.0.0)
```

Dataset Description

The dataset is taken from the UCI Machine Learning Repository.

It contains transactional data of an online retail store.

Features include:

- Customer ID
- Stock Code (Product ID)
- Quantity
- Invoice Date
- Price

```
# Load dataset (Upload file first in Colab)
data = pd.read_excel("/content/online_retail_II.xlsx", engine="openpyxl")
# Display first few rows
data.head()
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.0	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.0	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.0	United Kingdom

Data Understanding

We check dataset structure, columns, and size.

```
print("Shape:", data.shape)
print("Columns:", data.columns)

data.info()

Shape: (525461, 8)
Columns: Index(['Invoice', 'StockCode', 'Description', 'Quantity', 'InvoiceDate',
               'Price', 'Customer ID', 'Country'],
              dtype='object')
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 525461 entries, 0 to 525460
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Invoice          525461 non-null object
1   StockCode       525461 non-null object
2   Description     522533 non-null object
3   Quantity        525461 non-null int64
4   InvoiceDate     525461 non-null datetime64[ns]
5   Price          525461 non-null float64
6   Customer ID    417534 non-null float64
7   Country        525461 non-null object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 32.1+ MB
```

▼ Data Cleaning

- Remove missing Customer IDs
- Remove negative/zero quantities
- Remove duplicate records

```
# Remove missing Customer ID
data = data.dropna(subset=['Customer ID'])

# Remove invalid quantities
data = data[data['Quantity'] > 0]

# Remove duplicates
data = data.drop_duplicates()

print("After cleaning:", data.shape)
```

After cleaning: (400947, 8)

▼ Data Reduction

To avoid memory issues, we keep only top 1000 frequently purchased products.

```
top_products = data['StockCode'].value_counts().head(1000).index
data = data[data['StockCode'].isin(top_products)]
```

▼ Data Transformation

We convert data into a Customer-Product matrix.

```
basket = data.pivot_table(
    index='Customer ID',
    columns='StockCode',
    values='Quantity',
    aggfunc='sum',
    fill_value=0
)

basket.head()
```

StockCode	10002	10135	11001	15036	16237	17003	20658	20665	20668	20674	...	85206A	85206B	85226A	85231B	85231G	85232B	85232C
Customer ID																		
12346.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0
12347.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0
12348.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0
12349.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0
12351.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0

5 rows × 1000 columns

Binary Conversion

Convert values:

- 1 = Purchased
- 0 = Not Purchased

```

basket[basket > 0] = 1
basket[basket == 0] = 0

```

Filtering Customers

Remove customers with very few purchases to improve model learning.

```

basket = basket[basket.sum(axis=1) > 5]

```

Feature Scaling

Normalize data using MinMaxScaler.

```

# Fix column names (VERY IMPORTANT)
basket.columns = basket.columns.astype(str)

# Now apply scaling
scaler = MinMaxScaler()
basket_scaled = scaler.fit_transform(basket)

```

Training Boltzmann Machine (RBM)

RBM learns hidden patterns in customer purchase data.

```

rbm = BernoulliRBM(
    n_components=30,
    learning_rate=0.05,
    n_iter=20,
    random_state=0,
    verbose=True
)

rbm.fit(basket_scaled)

```

```

[BernoulliRBM] Iteration 1, pseudo-likelihood = -177.51, time = 0.45s
[BernoulliRBM] Iteration 2, pseudo-likelihood = -175.67, time = 0.55s
[BernoulliRBM] Iteration 3, pseudo-likelihood = -170.05, time = 1.10s
[BernoulliRBM] Iteration 4, pseudo-likelihood = -169.41, time = 2.01s
[BernoulliRBM] Iteration 5, pseudo-likelihood = -166.49, time = 1.31s
[BernoulliRBM] Iteration 6, pseudo-likelihood = -165.66, time = 0.78s
[BernoulliRBM] Iteration 7, pseudo-likelihood = -163.40, time = 0.76s
[BernoulliRBM] Iteration 8, pseudo-likelihood = -163.53, time = 0.88s
[BernoulliRBM] Iteration 9, pseudo-likelihood = -161.52, time = 0.86s
[BernoulliRBM] Iteration 10, pseudo-likelihood = -159.95, time = 1.14s
[BernoulliRBM] Iteration 11, pseudo-likelihood = -159.24, time = 1.00s
[BernoulliRBM] Iteration 12, pseudo-likelihood = -157.83, time = 0.41s
[BernoulliRBM] Iteration 13, pseudo-likelihood = -155.94, time = 0.42s
[BernoulliRBM] Iteration 14, pseudo-likelihood = -154.83, time = 0.41s
[BernoulliRBM] Iteration 15, pseudo-likelihood = -155.15, time = 0.40s
[BernoulliRBM] Iteration 16, pseudo-likelihood = -154.48, time = 0.45s
[BernoulliRBM] Iteration 17, pseudo-likelihood = -153.19, time = 0.66s
[BernoulliRBM] Iteration 18, pseudo-likelihood = -154.16, time = 0.47s
[BernoulliRBM] Iteration 19, pseudo-likelihood = -153.03, time = 0.41s
[BernoulliRBM] Iteration 20, pseudo-likelihood = -151.67, time = 0.41s

```

▾ BernoulliRBM ⓘ ?
 BernoulliRBM(learning_rate=0.05, n_components=30, n_iter=20, random_state=0, verbose=True)

Feature Extraction

Extract hidden features learned by RBM.

```
hidden = rbm.transform(basket_scaled)

print("Hidden shape:", hidden.shape)

Hidden shape: (3782, 30)
```

Customer Segmentation using KMeans

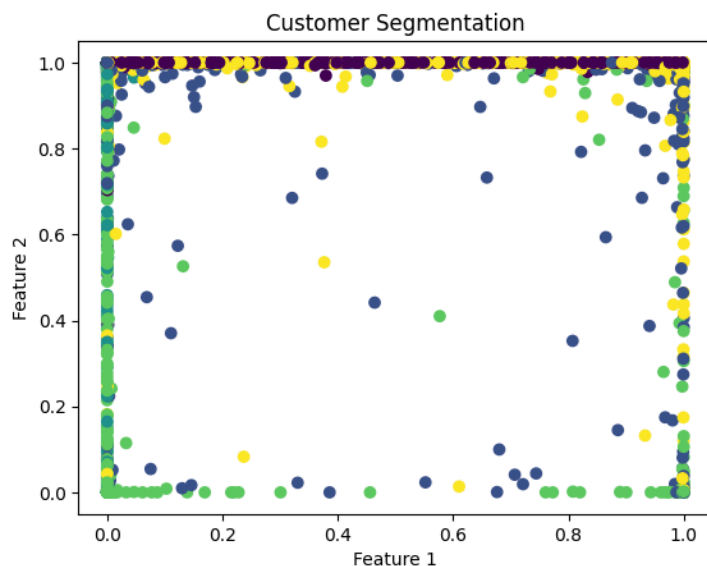
```
kmeans = KMeans(n_clusters=5, random_state=0)
clusters = kmeans.fit_predict(hidden)

print("Cluster labels:", clusters[:10])

Cluster labels: [0 4 4 1 0 0 0 0 1 3]
```

Visualization of Customer Segments

```
plt.figure()
plt.scatter(hidden[:, 0], hidden[:, 1], c=clusters)
plt.title("Customer Segmentation")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



Results

- RBM successfully learned hidden features
- Customers were grouped into clusters
- Similar purchasing behavior observed within clusters

Observations

- Customers with similar buying patterns are grouped together
- RBM captures hidden relationships between products
- Clustering improves segmentation clarity
- Data sparsity affects model performance

Conclusion

The Boltzmann Machine model successfully segmented customers based on purchasing behavior. This approach can be used in recommendation systems and targeted marketing.

